

House Price Prediction

Complete Data Analysis, Modeling & Recommendations Report
Ames, Iowa Housing Dataset

Source: generated from the executed analysis notebook *House_Price_Prediction.ipynb*, covering the Ames Housing dataset (Kaggle "House Prices – Advanced Regression Techniques") — 1,460 residential home sales in Ames, Iowa, described by 79 explanatory variables plus the target, **SalePrice**.

Scope of this report: (1) a complete data analysis / EDA report, (2) a supervised machine-learning model built to predict SalePrice and an analysis of which features drive price, (3) a comparison of multiple candidate models with a production recommendation, and (4) a summary of the data challenges encountered and how each was resolved.

Contents

- 1 Dataset Overview
- 2 Data Cleaning
- 3 Exploratory Data Analysis
- 4 Feature Engineering & Preprocessing
- 5 Model Selection & Evaluation
- 6 Hyperparameter Tuning
- 7 Model Comparison Report
- 8 Report on Challenges Faced

1. Dataset Overview

The dataset contains **1,460 rows and 81 columns** (79 features + Id + the target SalePrice). Of the 79 predictive features, **36 numerical** and **43 categorical** (after excluding Id and the target from the raw 38/43 split). **No duplicate rows** were found.

1.1 Target variable: SalePrice

Statistic	Value
Count	1,460
Mean	\$180,921
Std. deviation	\$79,443
Minimum	\$34,900
Median	\$163,000
Maximum	\$755,000
Skewness	1.883
Kurtosis	6.536

2. Data Cleaning

Nineteen columns contained missing values, ranging from a single missing Electrical record up to PoolQC, missing for 99.5% of homes. Per the data dictionary, most of this "missingness" is structural — it encodes that the feature does not exist for that house (no pool, no alley, no fireplace, etc.) rather than an unknown value.

2.1 Missing values before cleaning

Column	Missing count	Missing %
PoolQC	1,453	99.52
MiscFeature	1,406	96.30
Alley	1,369	93.77
Fence	1,179	80.75
MasVnrType	872	59.73
FireplaceQu	690	47.26
LotFrontage	259	17.74
GarageType / GarageYrBlt / GarageFinish / GarageQual / GarageCond	81	5.55
BsmtExposure / BsmtFinType2	38	2.60
BsmtQual / BsmtCond / BsmtFinType1	37	2.53
MasVnrArea	8	0.55
Electrical	1	0.07

2.2 Cleaning strategy applied

- **Categorical, structurally absent** (PoolQC, MiscFeature, Alley, Fence, FireplaceQu, GarageType/Finish/Qual/Cond, BsmtQual/Cond/Exposure/FinType1&2, MasVnrType) → filled with the explicit category 'None'.
- **Numeric, structurally absent** (GarageYrBlt, GarageArea, GarageCars, MasVnrArea, BsmtFinSF1/2, BsmtUnfSF, TotalBsmtSF, BsmtFullBath, BsmtHalfBath) → filled with 0.
- **LotFrontage** (missing without a "doesn't exist" interpretation) → imputed with the median LotFrontage of the same Neighborhood.
- **Electrical** (a single missing value) → imputed with the mode.

After cleaning, **zero missing values remain** (verified programmatically). The non-predictive Id column was dropped, leaving 80 columns for analysis.

3. Exploratory Data Analysis

3.1 Target variable distribution

SalePrice is right-skewed (skewness ≈ 1.88 , kurtosis ≈ 6.54): most homes cluster in the mid-price range with a long tail of expensive properties. A **log1p transform** was applied, reducing skewness to 0.121 and kurtosis to 0.810 — this log-transformed value was used as the modeling target throughout, with predictions back-transformed to dollars for reporting.

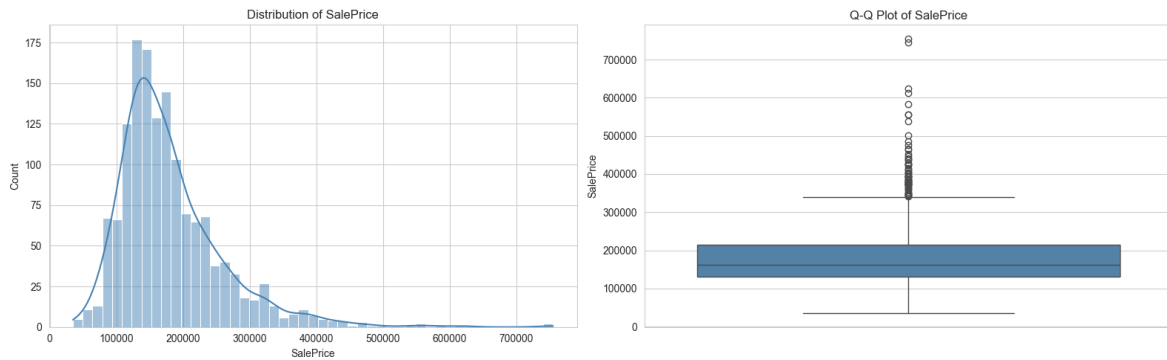


Figure 1. Distribution and spread of SalePrice (raw).

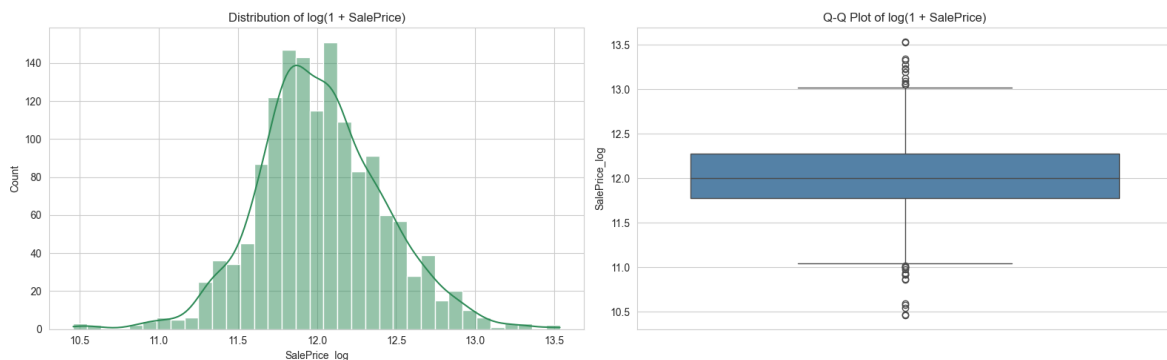


Figure 2. Distribution and spread of $\log(1+\text{SalePrice})$ — the modeling target.

3.2 Univariate analysis — numerical features

36 numerical features were reviewed for shape, skew, and "no feature" mass points at zero (e.g. 2ndFlrSF, PoolArea are 0 for most homes).

Distributions of Numerical Features

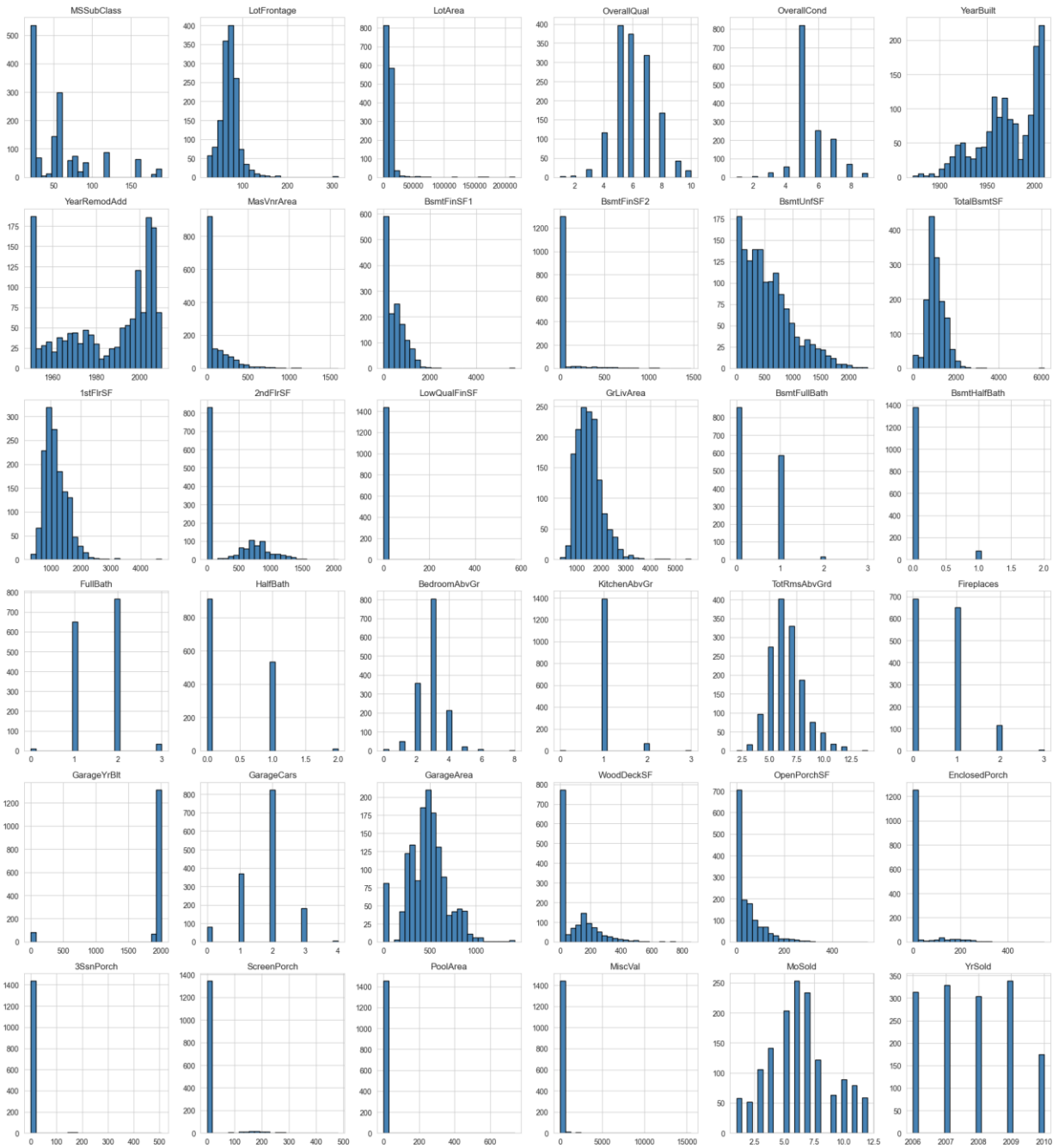


Figure 3. Distributions of all numerical features.

Boxplots of Key Numerical Features (outlier check)

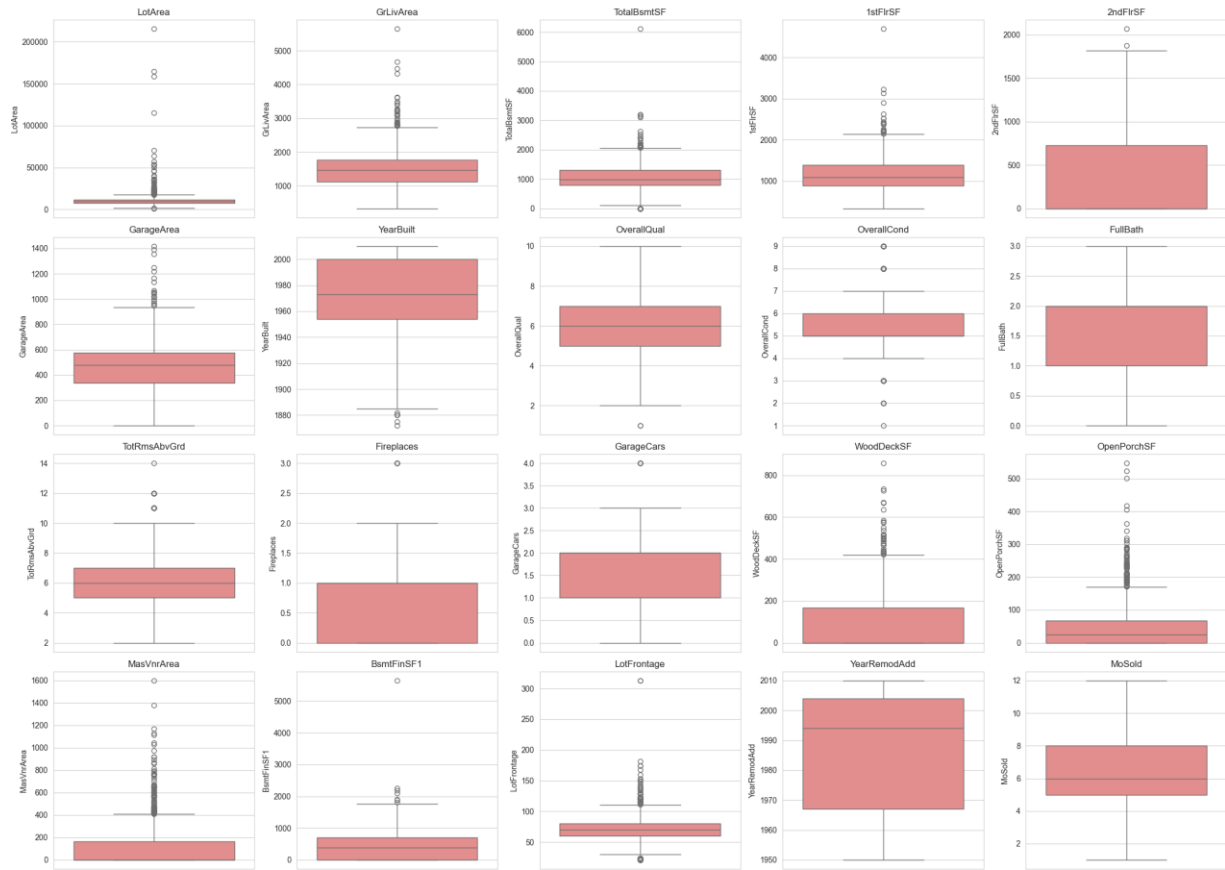
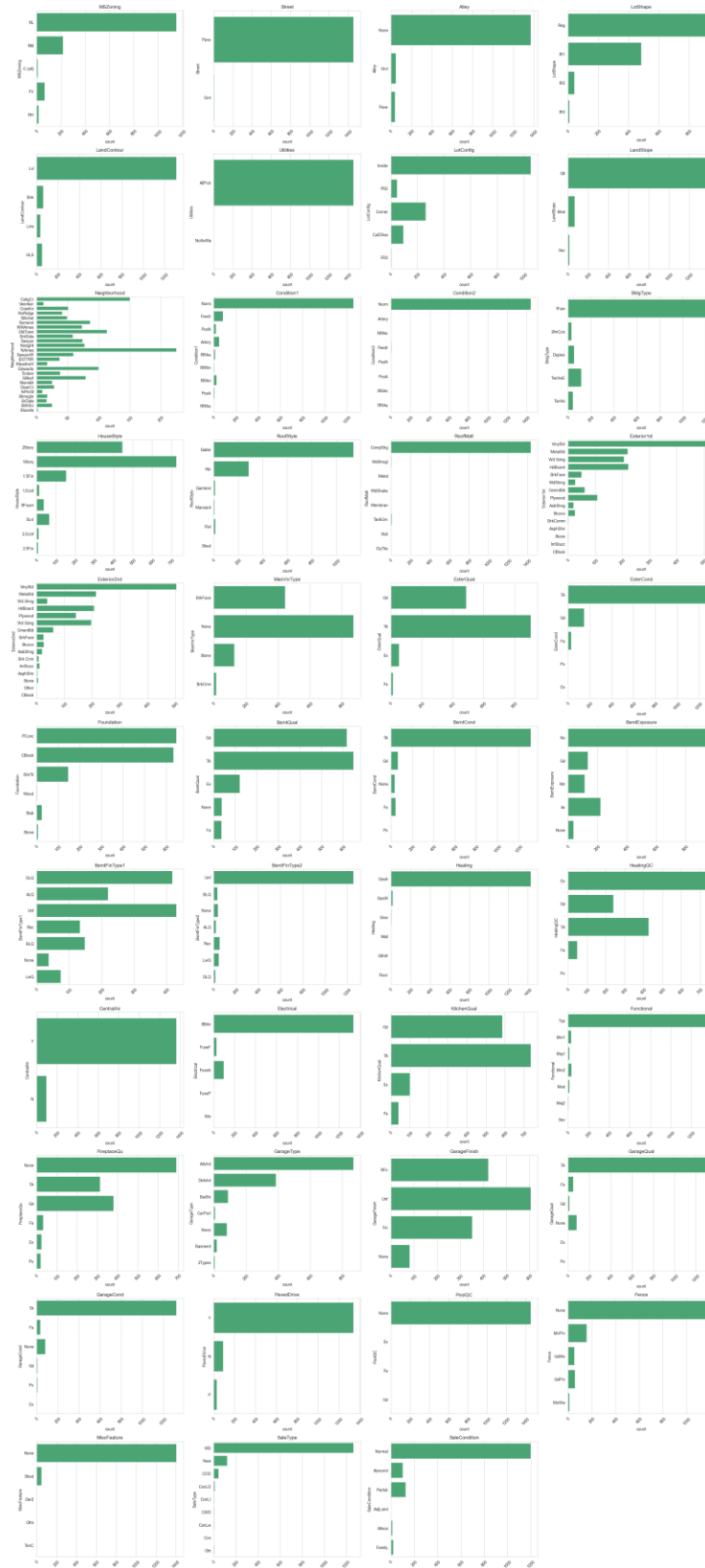


Figure 4. Boxplots of key numerical features, used to screen for outliers.

3.3 Univariate analysis — categorical features

Category frequencies were reviewed for all 43 categorical features. Several are highly imbalanced: Utilities is 99.9% a single value, Street is 99.6% "Pave" — these carry almost no predictive signal.



3.1 Correlation of numerical features with SalePrice

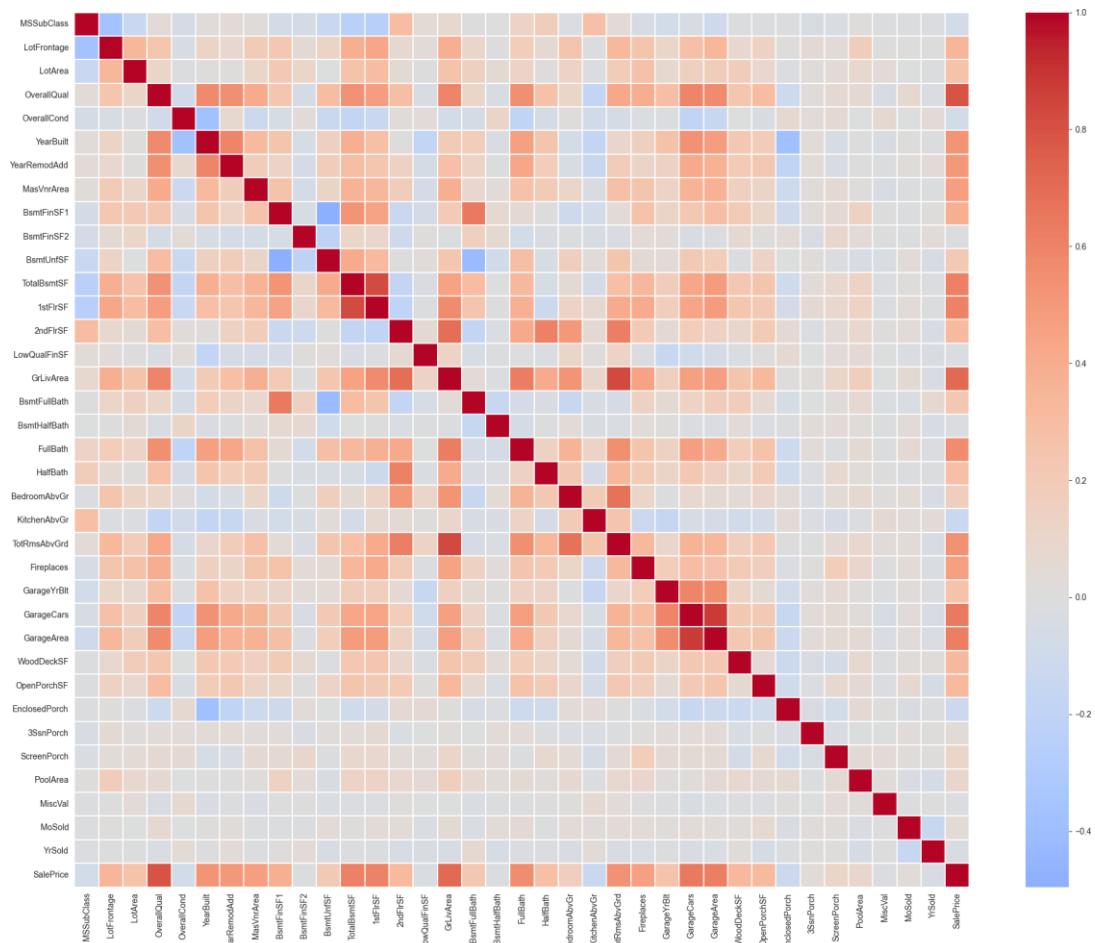


Figure 6. Correlation heatmap of numerical features.

Top positive correlations	r	Top negative correlations	r
OverallQual	0.791	EnclosedPorch	−0.129
GrLivArea	0.709	KitchenAbvGr	−0.136
GarageCars	0.640	MSSubClass	−0.084
GarageArea	0.623	OverallCond	−0.078
TotalBsmtSF	0.614	YrSold	−0.029
1stFlrSF	0.606	LowQualFinSF	−0.026
FullBath	0.561	MiscVal	−0.021
TotRmsAbvGrd	0.534	BsmtHalfBath	−0.017
YearBuilt	0.523	BsmtFinSF2	−0.011
YearRemodAdd	0.507	3SsnPorch	0.045

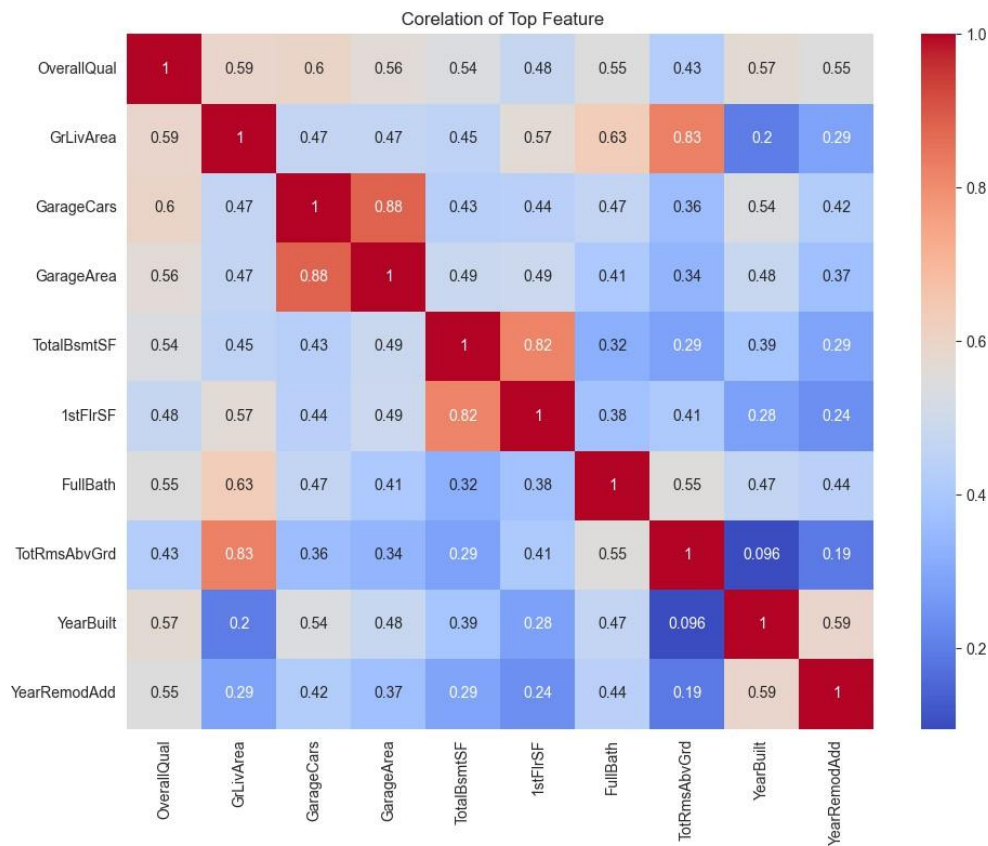


Figure 7. Correlation among the top 10 predictive features (multicollinearity check).

GarageCars/GarageArea and TotalBsmtSF/1stFlrSF are highly correlated with each other, as expected (both pairs describe the same physical quantity from slightly different angles). This matters for linear-model coefficient stability but is largely inconsequential for tree-based models.

3.2 Bivariate analysis — numerical vs. numerical

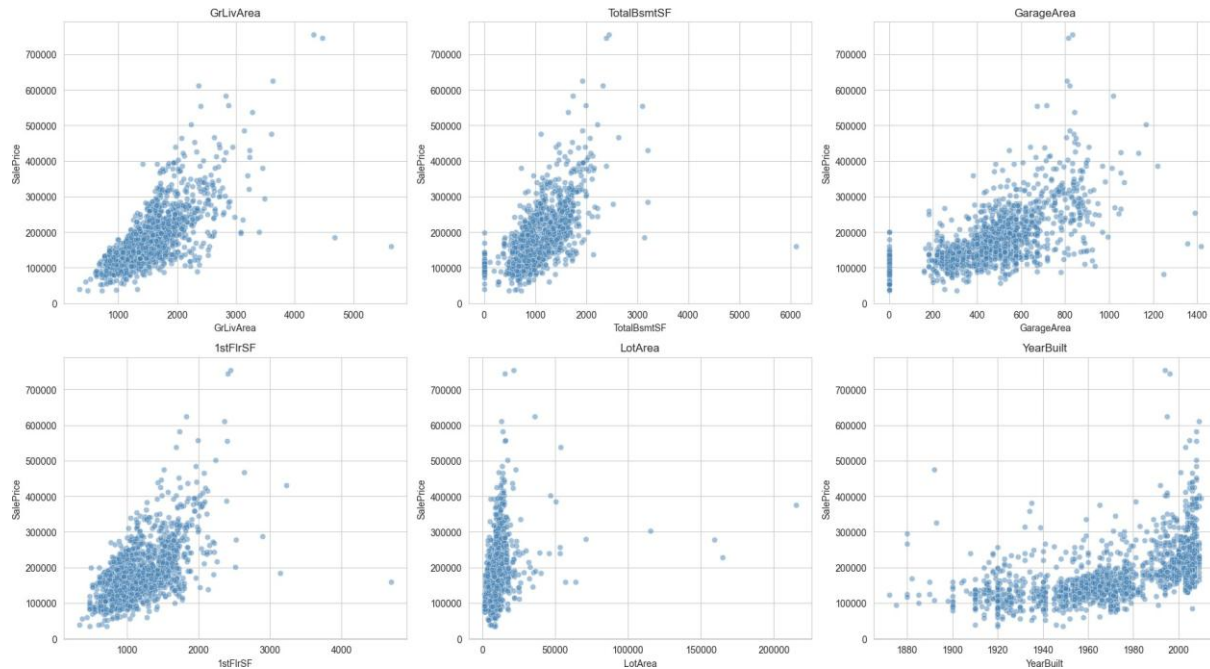


Figure 8. SalePrice vs. key numerical features.

Two homes with unusually large GrLivArea (>4,000 sq ft) but a comparatively low SalePrice (<\$300k) were identified as outliers — a pattern documented for this dataset. These two records were removed before modeling, leaving 1,458 rows.

3.3 Bivariate analysis — categorical vs. numerical

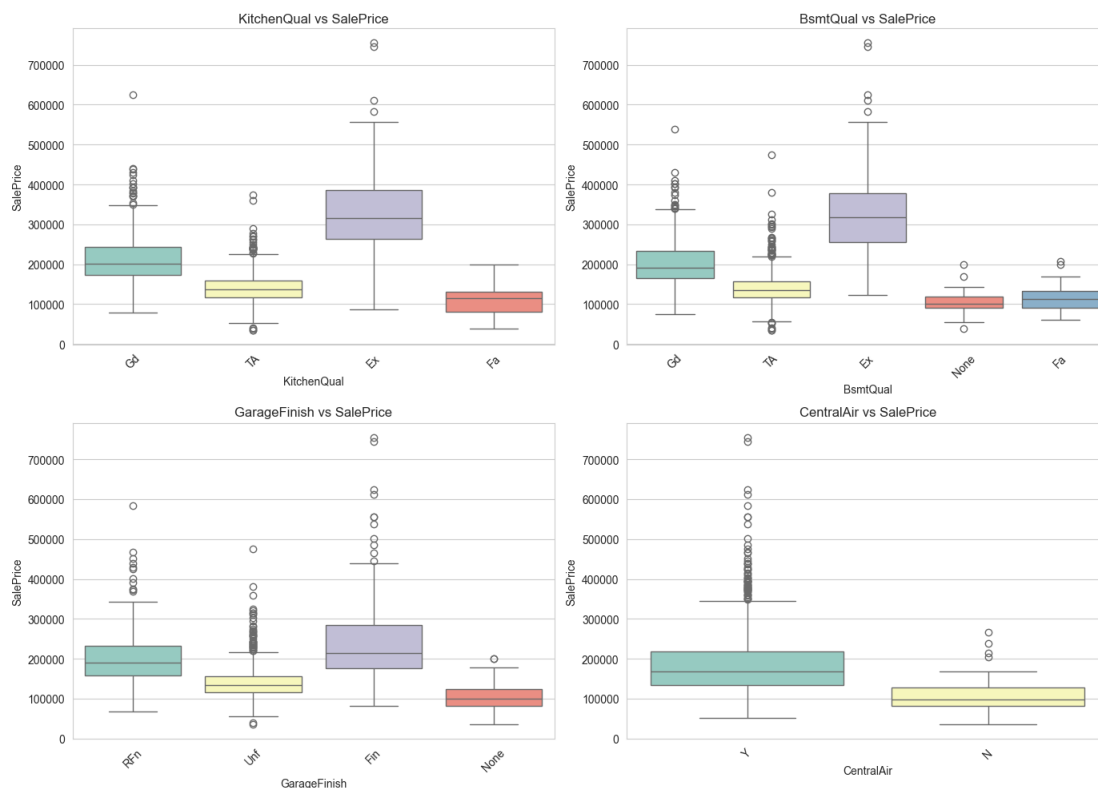


Figure 9. SalePrice by KitchenQual, BsmtQual, GarageFinish, and CentralAir.

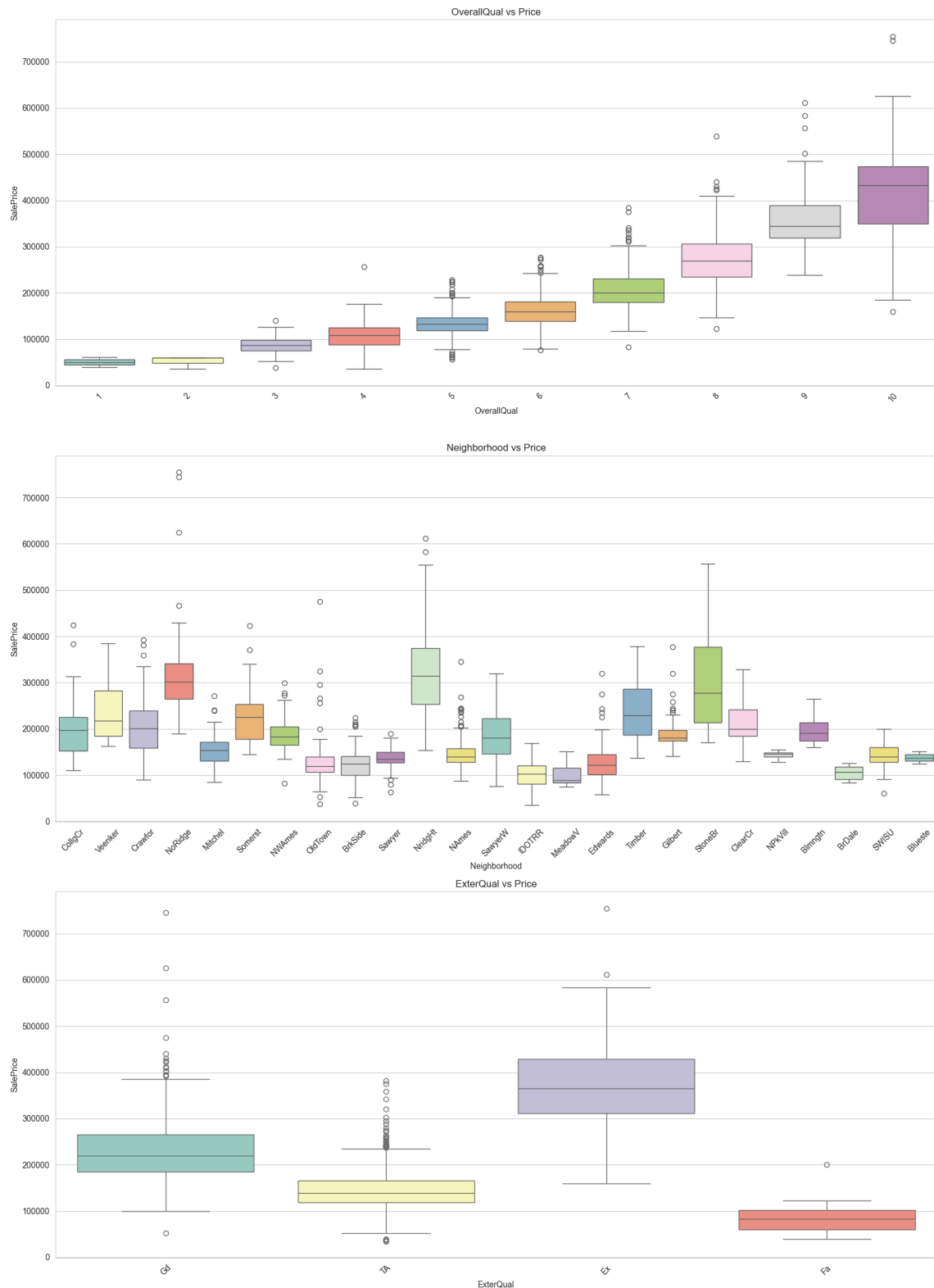


Figure 10. SalePrice by OverallQual, Neighborhood, and ExterQual.

Quality-related categorical features show a clean, roughly monotonic relationship with price, and Neighborhood shows substantial spread — location is a first-order price driver independent of the house's physical attributes.

3.4 Multivariate analysis

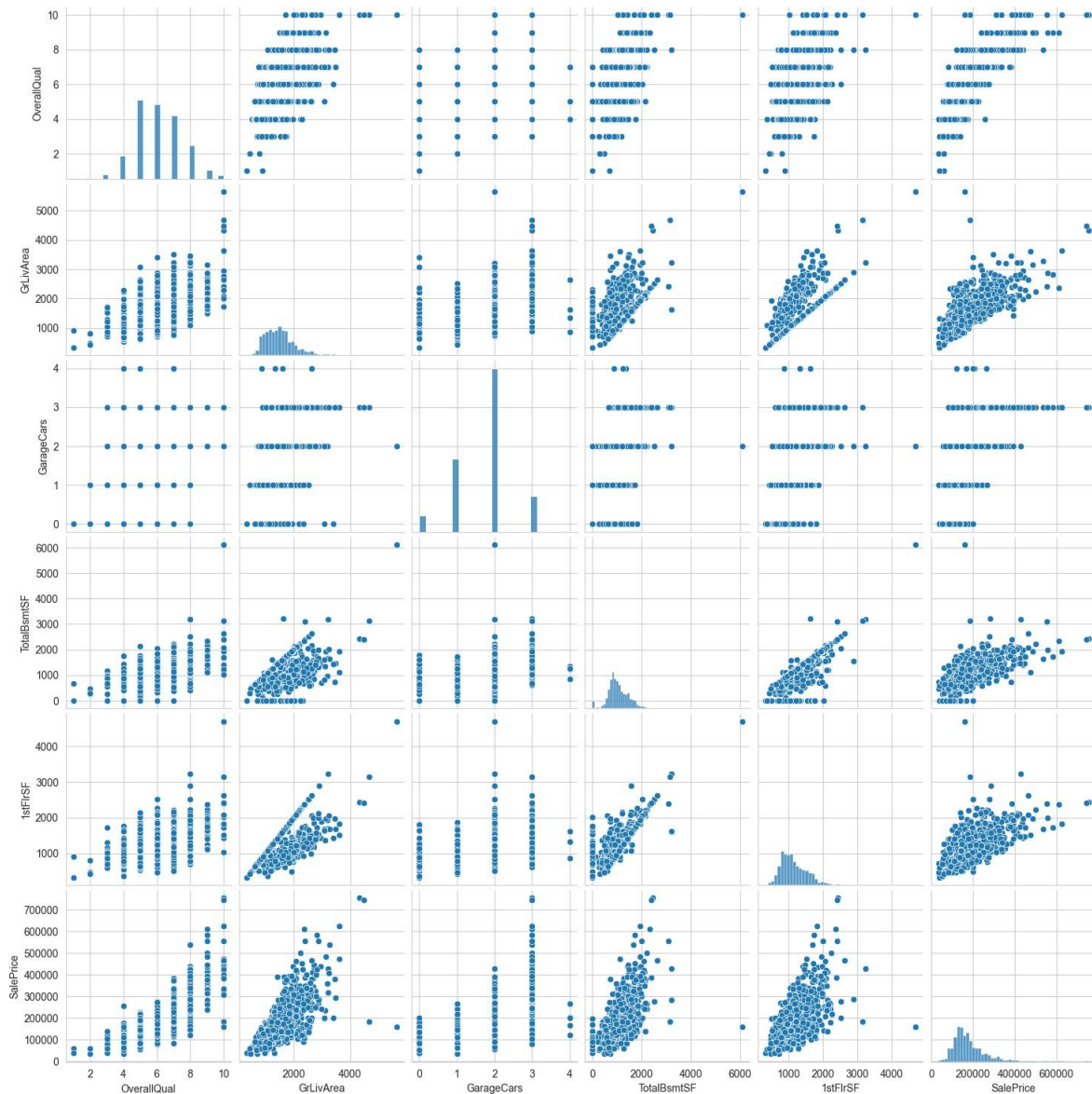


Figure 11. Pairwise relationships among OverallQual, GrLivArea, GarageCars, TotalBsmtSF, 1stFlrSF, and SalePrice.

No single feature explains price in isolation — the pairwise scatter matrix confirms that GrLivArea, OverallQual, GarageCars, and TotalBsmtSF interact rather than act independently on SalePrice.

3.5 EDA summary

- **SalePrice** is right-skewed → modeled on the log scale.
- **OverallQual, GrLivArea, GarageCars/GarageArea, TotalBsmtSF, YearBuilt** are the strongest single predictors.
- **Neighborhood** carries strong location-based price signal.
- Two extreme **outliers** (very large GrLivArea, unexpectedly low SalePrice) were identified and removed.
- Several categorical features (**Utilities, Street**) are near-constant and contribute little signal.

4. Feature Engineering & Preprocessing

4.1 Engineered features

- **HouseAge** = YrSold – YearBuilt
- **RemodAge** = YrSold – YearRemodAdd
- **TotalSF** = TotalBsmtSF + 1stFlrSF + 2ndFlrSF (total finished square footage)
- **TotalBath** = FullBath + 0.5×HalfBath + BsmtFullBath + 0.5×BsmtHalfBath
- Additional porch/pool/garage/fireplace presence and totals features
- Ordinal quality/condition text scales (Ex/Gd/TA/Fa/Po/None) mapped to numeric 0–5, preserving their natural order for the model

After feature engineering the working frame had **1,458 rows × 82 columns** (2 outlier rows removed, several engineered columns added).

4.2 Preprocessing pipeline

All preprocessing was wrapped in a scikit-learn ColumnTransformer + Pipeline to avoid data leakage — imputation and scaling statistics are fit only on the training fold:

- **Numerical features:** median imputation → standard scaling
- **Categorical features:** most-frequent imputation → one-hot encoding (unseen categories at inference time are safely ignored)

4.3 Train / test split

An 80/20 split was used: **1,166 training rows, 292 test rows**.

5. Model Selection & Evaluation

Eleven candidate regression algorithms were trained inside the same preprocessing pipeline and compared using 5-fold cross-validation on the training split. The evaluation metric is RMSE on the log-transformed target.

5.1 Cross-validation results (training data)

Model	CV RMSE (log)	CV Std	CV R ²
ElasticNet	0.1124	0.0098	0.9187
Ridge	0.1133	0.0105	0.9172
Lasso	0.1140	0.0092	0.9163
GradientBoosting	0.1199	0.0121	0.9078
XGBoost	0.1201	0.0123	0.9076
LinearRegression	0.1236	0.0141	0.9004
RandomForest	0.1351	0.0128	0.8831
SVR	0.1473	0.0114	0.8589
AdaBoost	0.1639	0.0103	0.8277
KNN	0.1650	0.0144	0.8251
DecisionTree	0.2012	0.0092	0.7368

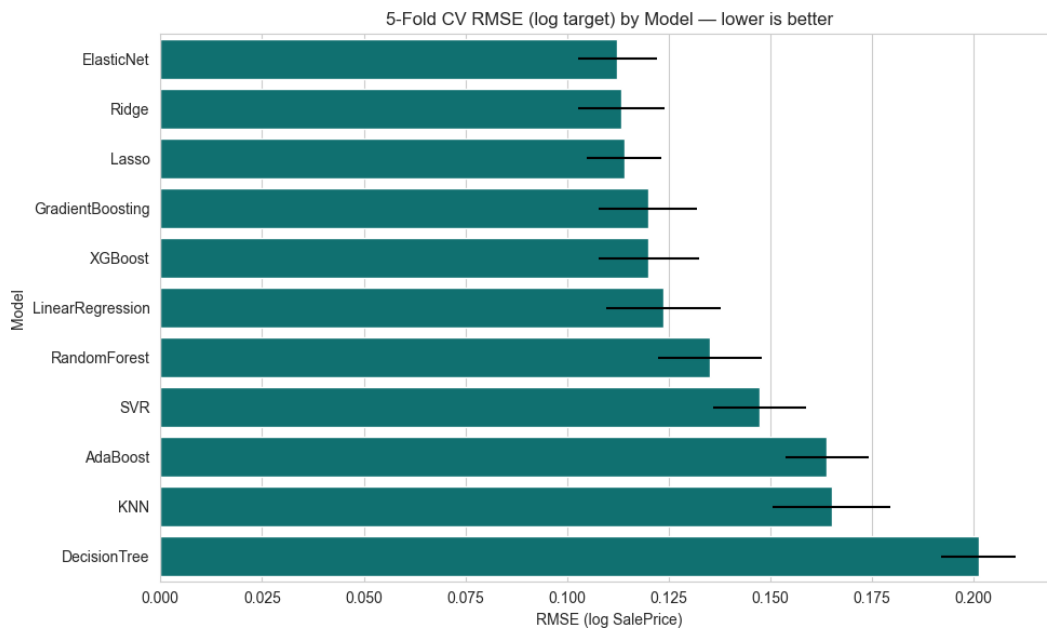


Figure 12. 5-fold cross-validation RMSE (log scale) by model — lower is better.

5.2 Held-out test-set evaluation (top 6 models)

Model	Test RMSE (log)	Test R ²	Test RMSE (\$)	Test MAE (\$)
ElasticNet	0.1174	0.9182	\$19,745	\$14,291

Model	Test RMSE (log)	Test R ²	Test RMSE (\$)	Test MAE (\$)
Lasso	0.1181	0.9172	\$19,390	\$14,097
GradientBoosting	0.1199	0.9147	\$19,462	\$13,988
XGBoost	0.1214	0.9126	\$19,525	\$14,214
Ridge	0.1215	0.9124	\$20,364	\$14,649
LinearRegression	0.1317	0.8971	\$22,378	\$15,752

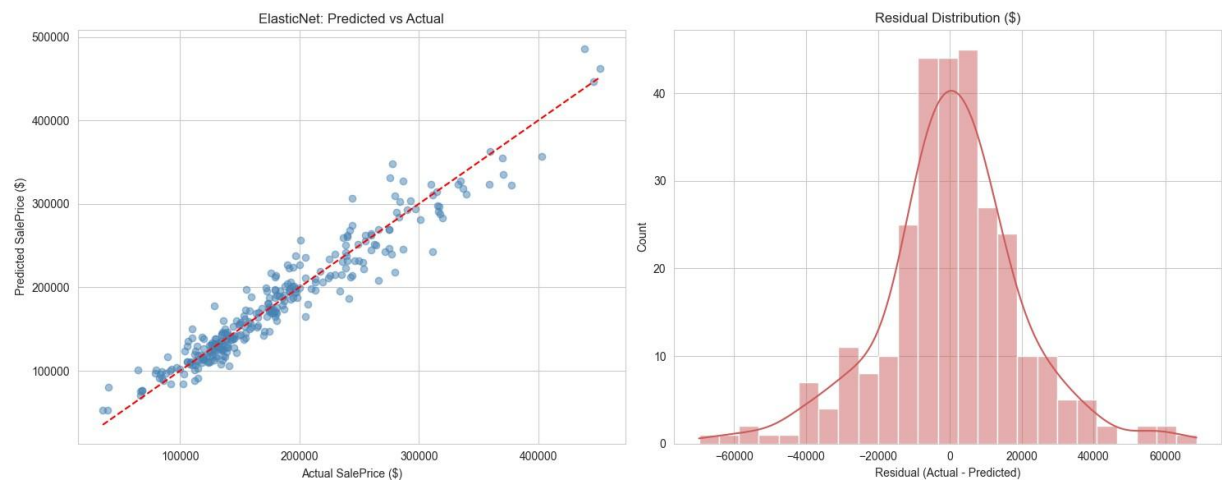


Figure 13. Best pre-tuning model (ElasticNet): predicted vs. actual SalePrice, and residual distribution.

6. Hyperparameter Tuning

RandomizedSearchCV (5-fold CV) was used to tune **XGBoost**, the top-performing boosting model, over `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, and `min_child_weight`.

Parameter	Tuned value
<code>n_estimators</code>	1000
<code>max_depth</code>	3
<code>learning_rate</code>	0.05
<code>subsample</code>	0.8
<code>colsample_bytree</code>	0.6
<code>min_child_weight</code>	1

	Test RMSE (log)	Test R ²	Test RMSE (\$)
XGBoost (before tuning)	0.1214	0.9126	\$19,525
XGBoost (after tuning)	0.1148	0.9219	\$18,334

Tuning improved XGBoost's held-out RMSE by roughly 6% and R² from 0.913 to 0.922 — enough to overtake every linear model and become the **best-performing model overall** on the held-out test set in this run.

7. Model Comparison Report

7.1 Final ranking (held-out test set, best to worst)

Rank	Model	Test RMSE (log)	Test R ²	Test RMSE (\$)
1	XGBoost (tuned)	0.1148	0.9219	\$18,334
2	ElasticNet	0.1174	0.9182	\$19,745
3	Lasso	0.1181	0.9172	\$19,390
4	GradientBoosting	0.1199	0.9147	\$19,462
5	XGBoost (untuned)	0.1214	0.9126	\$19,525
6	Ridge	0.1215	0.9124	\$20,364
7	LinearRegression	0.1317	0.8971	\$22,378

7.2 Interpretation by model family

Model family	Finding
Regularized linear (Ridge/Lasso/ElasticNet)	Fast, interpretable, and the strongest performers before tuning; ElasticNet/Lasso also perform implicit feature selection. Remains sensitive to residual multicollinearity/outliers.
Plain Linear Regression	Clearly weaker than the regularized variants — no protection against multicollinearity among the size-related features.
Single trees / KNN / Underfitting	Single tree over/underfits depending on depth, and KNN/AdaBoost struggle in this high-dimensional, mixed-type feature space.
Random Forest	Robust with little tuning but did not match boosting or the regularized linear models here.
Boosting (Gradient Boosting / XGBoost)	After tuning, XGBoost became the single best model on the held-out test set, capturing feature interactions (e.g. quality).

7.3 Recommendation for production

On this dataset and split, the **tuned XGBoost** model gave the best held-out accuracy, narrowly ahead of ElasticNet. Practical guidance by priority:

- **Raw accuracy priority** (automated valuation model) → **tuned XGBoost**, the top performer in this evaluation.
- **Interpretability priority** (explaining valuations to stakeholders/regulators) → **ElasticNet or Lasso** on the log target: coefficients are directly interpretable as multiplicative effects on price, and their accuracy is close to XGBoost's.
- **Balanced / low-maintenance choice** → **GradientBoosting**: strong accuracy with less tuning sensitivity than XGBoost.
- Whichever model ships to production should be **retrained periodically** as new sales data arrives, since housing markets shift in ways the training window cannot capture.
- The current notebook already **persists the best pipeline** to `../models/best_model.pkl` via pickle, which is a reasonable basis for a simple deployment.

8. Report on Challenges Faced

1. High proportion and structure of missing data

Challenge: PoolQC, MiscFeature, Alley, Fence, FireplaceQu, and the garage/basement quality fields had large fractions of missing values. Dropping these columns/rows would have discarded most of the dataset and real signal, since NaN in these columns encodes "feature does not exist" rather than "value unknown."

Technique used: Domain-informed imputation — 'None' for absent categorical features, 0 for the corresponding numeric features — rather than generic mean/mode imputation, which would have implied the feature existed at an average level.

2. LotFrontage missingness not tied to feature absence

Challenge: LotFrontage was missing for ~17.7% of rows without an obvious "does not exist" interpretation, since every house fronts some street.

Technique used: Imputed with the median LotFrontage of the house's own Neighborhood, since frontage follows local lot-layout conventions; a plain global median would have ignored this structure.

3. Skewed target variable

Challenge: SalePrice was right-skewed (skew ≈ 1.9), violating the normal-residual assumption linear models rely on and letting a few very expensive homes dominate squared-error loss.

Technique used: Modeled $\log_{10}(\text{SalePrice})$ throughout, back-transforming predictions with $\exp(m)$ for dollar-scale reporting — skewness dropped from 1.88 to 0.12 after the transform.

4. Outliers

Challenge: Two homes had very large GrLivArea but unexpectedly low SalePrice (documented anomalies for this dataset). Left in, they disproportionately pull linear-model fits.

Technique used: Identified via scatterplot against the target and removed the two clearest cases, rather than aggressively filtering on any single feature, to avoid discarding legitimate high-end sales.

5. High cardinality and imbalanced categoricals

Challenge: Neighborhood has 25 categories; features like Utilities (99.9% one value) and Street (99.6% one value) contribute little.

Technique used: One-hot encoding via ColumnTransformer (with `handle_unknown='ignore'` for robustness to unseen categories at inference time) for informative categoricals; near-constant columns were flagged during EDA.

6. Multicollinearity among size-related features

Challenge: GarageCars/GarageArea, and TotalBsmtSF/1stFlrSF are highly correlated with each other, inflating coefficient variance in linear models (though it barely affects tree-based models).

Technique used: Retained all features (tree models handle collinearity natively) but relied on Ridge/Lasso/ElasticNet regularization to stabilize the linear fits.

7. Risk of data leakage in preprocessing

Challenge: Median/mode imputation and feature scaling must be fit only on training data to give an honest estimate of test performance.

Technique used: Wrapped all preprocessing in a scikit-learn Pipeline/ColumnTransformer so fitting happens only inside cross-validation folds and on the training split, never on the full dataset before splitting.

8. Choosing an evaluation metric that matches the business goal

Challenge: Raw RMSE in dollars over-weights errors on expensive homes.

Technique used: Trained and primarily evaluated on log-scale RMSE, while also reporting back-transformed dollar RMSE/MAE for stakeholder-facing interpretability.

9. Balancing model complexity against interpretability

Challenge: The winning model (tuned XGBoost) is harder to explain to a non-technical buyer or stakeholder than a linear model's coefficients.

Technique used: Kept both an interpretable (ElasticNet/Lasso) and a high-accuracy (tuned XGBoost) model in the comparison so either can be chosen depending on whether accuracy or explainability matters more for a given use case.
